

Privacy-Preserving Face Recognition

using Federated Learning

Abderrahim Khefif Kossai Baha Afaf Khadraoui
Amel Saidouni Mehdi Bouzoul

Motivation & Problem Statement

o

The Central Problem

Face recognition is everywhere — smartphones, border control, workplaces.

Standard systems send photos to a **central server**: a single point of privacy failure.

Unlike a password, a face cannot be changed after a breach.

Our Answer: Federated Learning + a layered privacy stack

Core Privacy Guarantee

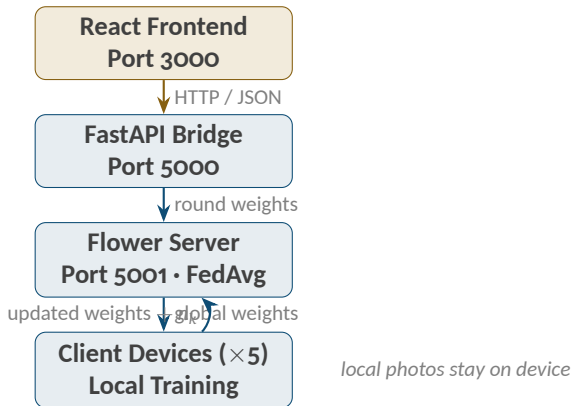
Raw face photographs **never leave** the client device.

Only **model weight updates** travel to the server.

We add formal DP, digital signatures, encryption, and audit logging.

System Architecture

o



The frontend is a **React web app**. FastAPI bridges the ML pipeline with the UI via `/api/register`, `/api/recognize`, and `/api/fl-results`.

Two-Model Strategy

o

Version A — Vulnerable Baseline

Federated Learning, **no DP**

Intentionally vulnerable to inversion

Establishes the attack surface

Version B — DP-Protected

- FL + **Opacus DP-SGD**
- $\sigma = 1.1, C = 1.0, \varepsilon_{\max} = 5.0$
- Identical architecture & rounds

Why Two Versions?

A privacy claim is only meaningful if the **baseline is actually vulnerable** and the protected model is **actually harder to attack**. The side-by-side inversion loss comparison is the core experimental contribution.

Parameter	Value
Rounds / Clients	20 / 5
Local Epochs	5
Learning Rate	10^{-4}
Attack Iterations	1000
Noise σ	1.1

Table of Contents

1 Pipeline

► Pipeline

► Privacy & Security

► Attack Evaluation

► Frontend & Limitations

► Conclusion

Step 1 – Face Detection & Preprocessing

1 Pipeline

MTCNN Three-Stage Cascade:

1. **P-Net** — multi-scale region proposals
2. **R-Net** — false positive rejection
3. **O-Net** — final box + 5 landmarks
(left/right eye, nose, mouth corners)

Blur pre-check: Laplacian variance $< 100 \Rightarrow$
image rejected.

*In practice: 2 of 3 photos passed; the third
required retaking.*

Processing Chain

- **Align:** rotate so both eyes are horizontal
- **Crop & Resize:** 160×160 px, 20 px margin
- **Normalise:** $x' = \frac{x-127.5}{128} \in [-1, 1]$
- **Save as .pt** (lossless 32-bit float)

Step 2 – Data Augmentation

1 Pipeline

Goal: Maximise training diversity from only 3 source photos per person.

Transform	Parameters	Simulates
Horiz. flip	$p = 0.5$	Left/right variation
Affine	$\text{rot } \pm 5^\circ$, scale $[0.95, 1.05]$	Head pose
Colour jitter	bright/contrast/sat. 0.2	Lighting
Hue jitter	hue 0.05	Colour cast

Applied **on-the-fly per batch** via
FaceAugmentation — no extra disk writes.

Effective Training Set

3 photos $\times$ 5 epochs $\approx$ **60**
distinct samples / person

Inference: raw normalised tensor only
— no augmentation, ensuring
embedding stability.

Step 3 – Feature Extraction

1 Pipeline

Model: InceptionResNetV1

Pretrained on VGGFace2 · 23M parameters

99.6% face verification accuracy on LFW

Model	Params	LFW Acc.
MobileNetV2	3.4M	~92%
InceptionResNetV1	23M	99.6%

Output: 512-dim L2-normalised vector

Mode: classify=False — raw descriptors

Recognition Geometry

Same person: Euclidean dist. < 0.6

Different people: dist. > 0.6

Recognition is distance comparison, not a classifier.

Enrollment & Recognition

1. 1-3 photos $\rightarrow$ embed $\rightarrow$ **avg template**
2. Live frame $\rightarrow$ embed $\rightarrow$ **cosine sim** vs. templates
3. Closest match below 0.6 $\Rightarrow$ identity

Step 4 – Federated Learning

1 Pipeline

FedAvg — One Round:

1. Server broadcasts global weights W_g
2. Each of 5 clients trains locally (5 epochs)
3. Clients return W_k + sample count n_k
4. Weighted average: $W_g = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} W_k$

Implementation

- **Framework:** Flower (flwr)
- **5 virtual clients** via Python multiprocessing
- Local socket 127.0.0.1:5001
- Adam, lr = 10^{-4}
- **Never sent:** photos, embeddings, names

Table of Contents

2 Privacy & Security

► Pipeline

► Privacy & Security

► Attack Evaluation

► Frontend & Limitations

► Conclusion

Privacy & Security Stack

2 Privacy & Security

FL alone is insufficient: model weights carry **statistical traces** exploitable by inversion and membership-inference attacks.

Defence 1: Differential Privacy

Per-sample gradient clipping ($C = 1.0$) + Gaussian noise ($\sigma = 1.1$). Formal (ϵ, δ) guarantee tracked by Opacus.

Defence 2: Ed25519 Signatures

Every client signs its update manifest (round, client ID, n_k , param hash). Server rejects invalid or missing signatures.

Defence 3: AES-256-GCM Encryption

Ephemeral X25519 key agreement + HKDF-derived key + AES-256-GCM. AAD binds round + client ID $\Rightarrow$ no replay. *Status: probe only — transport still plaintext.*

Defence 4: Hash-Chained Audit Logs

Each entry stores its own hash and the previous entry's hash. Editing any line breaks the chain.

Differential Privacy — Mechanism & Budget

2 Privacy & Security

Step 1 — Per-sample Gradient Clipping:

$$\tilde{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right), \quad C = 1.0$$

Step 2 — Gaussian Noise Injection:

$$\hat{g} = \frac{1}{B} \left(\sum_{i=1}^B \tilde{g}_i + \mathcal{N}(0, \sigma^2 C^2 I) \right)$$

$\sigma = 1.1$. No single photo can dominate the update.

Privacy Presets ($\delta = 10^{-5}$)

σ	Target ε	Strength
0.5	≤ 10	Weak
0.8	≤ 7	Moderate
1.1	≤ 5.0	Strong (used)
1.5	≤ 3	Very strong

Honest Result

Configured cap $\varepsilon_{\max} = 5.0$.

Logged cumulative $\varepsilon = \mathbf{10.21}$.

Hard-stop was not correctly wired.

Formal guarantee: $\varepsilon = 10.21$.

Technical Fix: Solving the Memory Crash

2 Privacy & Security

The Problem (OOM Error):

- Opacus DP-SGD stores ∇ for every parameter per sample.
- 23M params $\times$ 5 clients $\approx$ Crash.

The Solution:

- **Frozen Backbone:** Only train the projection head (260K params).
- **Manual DP Module:** Lightweight clipping/noising loop.

From-Scratch DP Logic

```
2. Gaussian Noising for p in params: noise =  
    torch.randn_like(p.grad) * (sigma * max_norm)p.grad.add_(noise)  
3. Standard Update self.optimizer.step()
```

Security: Ed25519 Signatures & AES-256-GCM

2 Privacy & Security

Ed25519 Signing Workflow

1. Client builds **manifest**:
round · client ID · n_k · param hash · protocol version
2. Client **signs** with Ed25519 private key
3. Server **verifies** signature
4. Reject if invalid, malformed, or wrong version
5. Event recorded in **audit log**

Protects *integrity* and *authenticity* of every update.

AES-256-GCM Encryption Probe

- Ephemeral **X25519** key agreement per session
- **HKDF**-derived symmetric key
- **AES-256-GCM** encrypts payload
- AAD binds round + client ID — prevents **replay attacks**

Current Status

Primitives **implemented and unit-tested**. Flower transport still **plaintext**. Wiring into serialisation layer is the planned next step.

Table of Contents

3 Attack Evaluation

► Pipeline

► Privacy & Security

► **Attack Evaluation**

► Frontend & Limitations

► Conclusion

Model Inversion Attack — Methodology

3 Attack Evaluation

Threat model: Attacker holds only the released global weights and a target embedding.
No training photos, no enrolment data.

Attack Loop (1000 iterations):

1. Initialise $\mathbf{x}$ as random 160×160 noise
2. Forward pass through **frozen** model f_θ
3. Compute pixel-space loss: $\mathcal{L} = \|f_\theta(\mathbf{x}) - \mathbf{e}_{\text{target}}\|^2$
4. Backprop w.r.t. *pixels only* (Adam, lr = 0.01)
5. Repeat — model weights **never updated**

Why Loss, Not Appearance?

Version A: clean gradients $\Rightarrow$ optimiser converges to low loss.

Version B (DP): noisy gradient landscape $\Rightarrow$ optimiser **hard-plateaus**.

Evidence lives in the **loss curve**.

Model Inversion — Results

3 Attack Evaluation

200-Iteration Convergence:

- **Version A (no DP):** loss $10^0 \rightarrow \approx 10^{-4}$
Attacker successfully approximates target embedding
- **Version B (DP):** early drop then **hard-plateau** at $\approx 10^{-2}$
- **Gap:** ≈ 2 orders of magnitude

Identity	Ver. A	Ver. B
Aaron Peirsol	2.29×10^{-4}	2.26×10^{-2}
Abdel N. Assidi	1.17×10^{-4}	1.94×10^{-2}

Extended 1000-Iter Run (Ver. A)

Final loss $\approx 1.60 \times 10^{-7}$

The 200-iter window **understates** the full convergence depth — Version A was still improving.

Conclusion

DP model is $\sim 100 \times$ **harder to optimise against**. Separation is consistent across both identities.

Table of Contents

4 Frontend & Limitations

► Pipeline

► Privacy & Security

► Attack Evaluation

► **Frontend & Limitations**

► Conclusion

Frontend & API Integration

4 Frontend & Limitations

React (Port 3000) + **FastAPI** (Port 5000)

Core API Endpoints:

- `/api/register` — MTCNN + embed + store template
- `/api/recognize` — cosine similarity vs. templates, threshold 0.6
- `/api/fl-results` — training metadata for dashboard

Two Privacy Modes:

Mode 1	Embeddings on server, recognition server-side
Mode 2	Embeddings local, recognition on-device

Actively Tested & Working

- Enrollment via live camera
- Live face recognition with overlay
- FL training dashboard (round progress)
- Attack comparison display

Mode 2 is the stronger guarantee: no biometric data leaves the device after training.

Known Limitations (1/2)

4 Frontend & Limitations

Backbone Freeze — FL Does Not Learn

Running 5 Opacus-wrapped Flower clients simultaneously exceeded available RAM.

Backbone frozen $\Rightarrow$ FL aggregation is **structurally operational but functionally inert**.

Partial fix implemented: a **scratch DP module** (embedding-level local DP + manual gradient clipping) was built to bypass Opacus memory overhead — reducing trainable params to $\sim 260\text{K}$ (head only, $\sim 100\times$ reduction). Full unfreeze still requires GPU.

ϵ Cap Not Enforced

Configured $\epsilon_{\max} = 5.0$; logged $\epsilon = 10.21$.
Training loop was not wired to halt at budget exhaustion.

Formal guarantee: $\epsilon = 10.21$, **not 5.0**.

Fix implemented: PrivacyMonitor raises PrivacyBudgetExceeded and halts training when the cap is crossed; warns at 80% of budget.

Known Limitations (2/2)

4 Frontend & Limitations

AES Not Connected to Transport

AES-256-GCM implemented and unit-tested.
Flower payloads travel in **plaintext** until the serialisation layer is wired.

No Per-Round Global Accuracy

Flower `History` object not serialised to disk.
Global FL accuracy curve cannot be reconstructed from existing data.

SecAgg+ Pending Migration

SecAgg+ requires Flower's newer `ServerApp/ClientApp` API. Current codebase uses the legacy strategy interface.

Migration path is identified but incomplete — emits a deprecation warning instead of a stronger guarantee.

Table of Contents

5 Conclusion

▶ Pipeline

▶ Privacy & Security

▶ Attack Evaluation

▶ Frontend & Limitations

▶ Conclusion

Conclusion & Future Work

5 Conclusion

What We Built:

- End-to-end federated face recognition system
- MTCNN + InceptionResNetV1 (VGGFace2)
- Flower FedAvg (20 rounds, 5 virtual clients)
- Opacus DP-SGD with privacy accounting
- Ed25519 signatures per update manifest
- AES-256-GCM + X25519 encryption probe
- Hash-chained tamper-evident audit logs
- React + FastAPI frontend, two privacy modes

Key Results

DP model is $\sim 100\times$ harder to invert, consistently across two identities.

Conclusion & Future Work

5 Conclusion

Future Work

1. **Unfreeze backbone** — GPU or sequential client execution to enable real FL learning
2. **Wire ϵ hard-stop** — PrivacyBudgetExceeded already implemented; needs wiring into FL loop
3. **Connect AES to transport** — encrypt Flower payloads end-to-end
4. **Complete SecAgg+** — migrate to new Flower ServerApp API
5. **Log per-round accuracy** — serialise Flower History object to disk
6. **Test FedProx** — better non-IID data handling vs. FedAvg

Privacy-Preserving Face Recognition

Thank you for listening!
Any questions?